

Simple Quantum Algorithms I

QML

Contents

1	Why Study These Algorithms?	3
2	The Oracle Model	3
2.1	What is an Oracle?	3
2.2	Phase Oracle	4
3	Deutsch's Algorithm	4
3.1	The Problem	4
3.2	The Circuit	4
3.3	Step-by-Step Analysis	4
3.4	Worked Example	5
4	Deutsch-Jozsa Algorithm	5
4.1	The Problem	5
4.2	The Circuit	6
4.3	Analysis	6
4.4	Oracle Construction Example	7
5	Bernstein-Vazirani Algorithm	7
5.1	The Problem	7
5.2	The Circuit	7
5.3	Analysis	8
6	Simon's Algorithm	8
6.1	The Problem	8
6.2	The Circuit	9
6.3	Analysis	9
6.4	Classical Post-Processing	9
6.5	Why Simon's Algorithm Matters	10
7	Comparison of the Algorithms	10
8	The Hadamard Transform and Fourier Analysis	11

8.1	The Boolean Fourier Expansion	11
9	Detailed Worked Examples	12
9.1	Deutsch-Jozsa with $n = 3$	12
9.2	Bernstein-Vazirani with $n = 4$	12
9.3	Simon's Algorithm: Full 3-Qubit Example	12
10	Limitations and Context	13
11	Summary	13

1 Why Study These Algorithms?

the algorithms in this lecture — deutsch, deutsch-jozsa, bernstein-vazirani, and simon's — r not practically useful. nobody is running deutsch's algorithm on a real quantum computer to solve a real problem. but they r pedagogically essential bcs they demonstrate, in the simplest possible settings, how quantum computers can solve certain problems faster than any classical computer.

each algorithm follows the same pattern:

1. create a superposition
2. query an oracle (a black-box function)
3. use interference to extract global information abt the function
4. measure to get the answer

this query-then-interfere pattern is the foundation of all quantum speedups. understanding it here makes grover's algorithm, shor's algorithm, and even QML feature maps much more intuitive.

Key Point. quantum algorithms exploit interference to amplify correct answers and cancel wrong ones. the art is in designing the right interference pattern for ur problem.

2 The Oracle Model

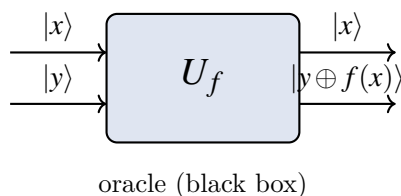
2.1 What is an Oracle?

Definition 2.1. an oracle (or black box) for a function $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$ is a unitary U_f that computes f reversibly:

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$$

where $\oplus$ is bitwise XOR (addition mod 2).

the oracle model is used for proving quantum speedups bcs it abstracts away the internal structure of f . u can only learn abt f by querying the oracle, and we count the number of queries needed.



Intuition. think of the oracle as a black box machine. u feed it an input, it gives u an output, and u r charged per query. the question is: how many queries do u need to figure out some property of f ? classical computers query with one input at a time. quantum computers can query with a superposition of all inputs simultaneously — thats where the speedup comes from.

2.2 Phase Oracle

theres an alternative oracle formulation thats super useful. if $f : \{0,1\}^n \rightarrow \{0,1\}$ (single-bit output), u can convert the standard oracle into a phase oracle:

$$\boxed{O_f |x\rangle = (-1)^{f(x)} |x\rangle}$$

how? set the ancilla to $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$:

$$U_f |x\rangle |-\rangle = |x\rangle \frac{1}{\sqrt{2}}(|0 \oplus f(x)\rangle - |1 \oplus f(x)\rangle) \quad (1)$$

$$= (-1)^{f(x)} |x\rangle |-\rangle \quad (2)$$

the phase $(-1)^{f(x)}$ gets “kicked back” onto the input register, and the ancilla remains $|-\rangle$. this is the phase kickback trick from lecture 2 in action.

3 Deutsch’s Algorithm

3.1 The Problem

given a function $f : \{0,1\} \rightarrow \{0,1\}$ (one bit to one bit), determine whether f is constant ($f(0) = f(1)$) or balanced ($f(0) \neq f(1)$).

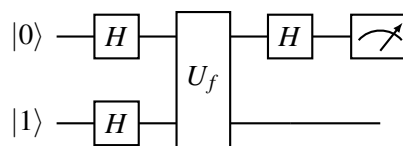
there r four possible functions:

x	f_1 (const 0)	f_2 (const 1)	f_3 (balanced)	x	f_4 (balanced)
0	0	1	0	0	1
1	0	1	1	1	0

classically: u need 2 queries (evaluate $f(0)$ and $f(1)$, then compare).

quantumly: 1 query suffices.

3.2 The Circuit



3.3 Step-by-Step Analysis

initial state: $|0\rangle |1\rangle$

after H on both qubits:

$$|+\rangle |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \cdot \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

after oracle U_f (using phase kickback):

$$\frac{1}{\sqrt{2}}((-1)^{f(0)} |0\rangle + (-1)^{f(1)} |1\rangle) |-\rangle$$

factor out $(-1)^{f(0)}$:

$$(-1)^{f(0)} \frac{1}{\sqrt{2}} (|0\rangle + (-1)^{f(0) \oplus f(1)} |1\rangle) |-\rangle$$

now there are two cases:

- if f is constant: $f(0) \oplus f(1) = 0$, so the state is $\pm |+\rangle |-\rangle$
- if f is balanced: $f(0) \oplus f(1) = 1$, so the state is $\pm |-\rangle |-\rangle$

after H on first qubit:

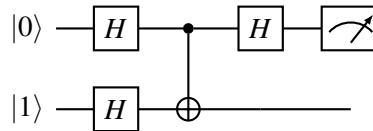
- constant: $H|+\rangle = |0\rangle \Rightarrow$ measure 0
- balanced: $H|-\rangle = |1\rangle \Rightarrow$ measure 1

Key Point. deutsch's algorithm determines whether f is constant or balanced with just 1 query to the oracle, compared to 2 classically. its only a factor-of-2 speedup, which is unimpressive. but the principle — query in superposition, use interference to extract a global property — generalizes to exponential speedups in the next algorithms.

3.4 Worked Example

Example 3.1. let $f(0) = 0, f(1) = 1$ (balanced, $f(x) = x$, i.e., the identity function).

the oracle is $U_f |x\rangle |y\rangle = |x\rangle |y \oplus x\rangle$, which is just a CNOT gate.



trace through:

$$|01\rangle \xrightarrow{H \otimes H} |+\rangle |-\rangle \tag{3}$$

$$\xrightarrow{\text{CNOT}} \frac{1}{\sqrt{2}} (|0\rangle |-\rangle + |1\rangle (X|-\rangle)) = \frac{1}{\sqrt{2}} (|0\rangle |-\rangle - |1\rangle |-\rangle) = |-\rangle |-\rangle \tag{4}$$

$$\xrightarrow{H \otimes I} |1\rangle |-\rangle \tag{5}$$

measure first qubit: get 1. conclusion: f is balanced. correct!

4 Deutsch-Jozsa Algorithm

4.1 The Problem

this is the natural generalization of deutsch's problem to n bits.

given $f : \{0,1\}^n \rightarrow \{0,1\}$ with the promise that f is either:

- constant: $f(x) = c$ for all x (same output for every input)

- balanced: $f(x) = 0$ for exactly half the inputs and $f(x) = 1$ for the other half

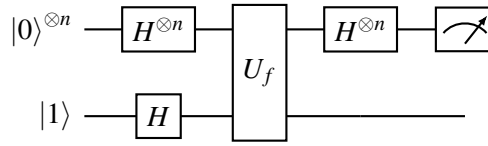
determine which case it is.

classically (deterministic): in the worst case, u need to query $2^{n-1} + 1$ inputs. bcs even after seeing 2^{n-1} identical outputs, the function could still be balanced (the remaining half could all be different).

classically (randomized): with high probability, $O(1)$ random queries suffice. if u query a few random inputs and they all give the same output, its very likely constant. but theres always a small probability of error.

quantumly: exactly 1 query, deterministic. no error.

4.2 The Circuit



4.3 Analysis

after first layer of hadamards:

$$\frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \otimes |-\rangle$$

after oracle (phase kickback):

$$\frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} (-1)^{f(x)} |x\rangle \otimes |-\rangle$$

after second layer of hadamards on the first register. we need the n-qubit hadamard identity:

$$H^{\otimes n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} (-1)^{x \cdot y} |y\rangle$$

where $x \cdot y = \sum_i x_i y_i \pmod{2}$ is the bitwise inner product.

so:

$$\frac{1}{\sqrt{2^n}} \sum_x (-1)^{f(x)} \cdot \frac{1}{\sqrt{2^n}} \sum_y (-1)^{x \cdot y} |y\rangle \quad (6)$$

$$= \sum_y \left[\frac{1}{2^n} \sum_x (-1)^{f(x) + x \cdot y} \right] |y\rangle \quad (7)$$

the amplitude of $|0^n\rangle$ (all zeros) is:

$$\alpha_{0^n} = \frac{1}{2^n} \sum_{x=0}^{2^n-1} (-1)^{f(x)}$$

- if f is constant: $\alpha_{0^n} = \frac{1}{2^n} \cdot (\pm 2^n) = \pm 1$. probability of measuring $|0^n\rangle$ is $\boxed{1}$.

- if f is balanced: $\alpha_{0^n} = \frac{1}{2^n}(2^{n-1} \cdot 1 + 2^{n-1} \cdot (-1)) = 0$. probability of measuring $|0^n\rangle$ is $\boxed{0}$.

Key Point. the algorithm: measure the first register. if u get 0^n , the function is constant. if u get anything else, its balanced. one query, deterministic, no error. this is an exponential speedup over deterministic classical algorithms.

Intuition. why does this work? the hadamard-oracle-hadamard structure creates interference. for a constant function, all the phases $(-1)^{f(x)}$ r the same, so they constructively interfere at $|0^n\rangle$. for a balanced function, half the phases r $+1$ and half r -1 , so they perfectly cancel at $|0^n\rangle$ (destructive interference). the interference pattern encodes the global property (constant vs balanced) into a single measurement outcome.

4.4 Oracle Construction Example

Example 4.1. for $n = 2$, consider the balanced function $f(00) = 0, f(01) = 1, f(10) = 1, f(11) = 0$. the phase oracle applies $(-1)^{f(x)}$:

$$O_f = \text{diag}(1, -1, -1, 1)$$

this can be decomposed as: $O_f = CZ_{12} \cdot (I \otimes Z)$ or equivalently using the fact that $(-1)^{x_1 \oplus x_2}$ where $f(x_1 x_2) = x_1 \oplus x_2$. the oracle is a CNOT followed by a Z on the second qubit (in the phase oracle formulation).

trace through the full algorithm:

$$|00\rangle \xrightarrow{H^{\otimes 2}} \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle) \quad (8)$$

$$\xrightarrow{O_f} \frac{1}{2}(|00\rangle - |01\rangle - |10\rangle + |11\rangle) \quad (9)$$

$$\xrightarrow{H^{\otimes 2}} |11\rangle \quad (10)$$

measure: $|11\rangle \neq |00\rangle$, so f is balanced. correct!

5 Bernstein-Vazirani Algorithm

5.1 The Problem

given an oracle for $f: \{0, 1\}^n \rightarrow \{0, 1\}$ with the promise that $f(x) = s \cdot x \pmod{2}$ for some unknown “secret string” $s \in \{0, 1\}^n$, find s .

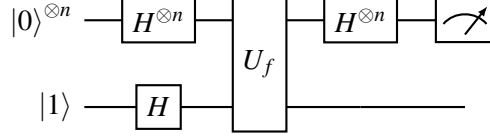
here $s \cdot x = s_1 x_1 \oplus s_2 x_2 \oplus \dots \oplus s_n x_n$ is the bitwise inner product mod 2.

classically: u need n queries. query $f(e_i)$ for each standard basis vector $e_i = 00 \dots 010 \dots 0$ (1 in position i). then $f(e_i) = s_i$, revealing one bit of s per query.

quantumly: 1 query.

5.2 The Circuit

the circuit is identical to deutsch-jozsa:



5.3 Analysis

after H, oracle (phase kickback), and H:

$$\sum_y \left[\frac{1}{2^n} \sum_x (-1)^{s \cdot x + x \cdot y} \right] |y\rangle = \sum_y \left[\frac{1}{2^n} \sum_x (-1)^{x \cdot (s \oplus y)} \right] |y\rangle$$

the inner sum is:

$$\frac{1}{2^n} \sum_x (-1)^{x \cdot (s \oplus y)} = \begin{cases} 1 & \text{if } s \oplus y = 0^n \text{ (i.e., } y = s) \\ 0 & \text{otherwise} \end{cases}$$

this uses the hadamard identity: $\sum_x (-1)^{x \cdot z} = 2^n \delta_{z, 0^n}$.

so the final state is simply $|s\rangle$. measure and u get the secret string directly.

Key Point. bernstein-vazirani achieves an exponential speedup: 1 quantum query vs n classical queries. the algorithm reveals all n bits of the secret string simultaneously by querying the oracle with a superposition of all possible inputs.

Example 5.1. $n = 3$, secret string $s = 101$.

$f(x) = x_1 \oplus x_3$ (inner product of x with 101).

the oracle adds a phase $(-1)^{x_1 \oplus x_3}$. after H-oracle-H:

final state: $|101\rangle$.

measure: 101. we recover $s = 101$ in one shot.

6 Simon's Algorithm

6.1 The Problem

given an oracle for $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$ with the promise that there exists a “period” $s \in \{0, 1\}^n$ such that:

$$f(x) = f(y) \iff x \oplus y \in \{0^n, s\}$$

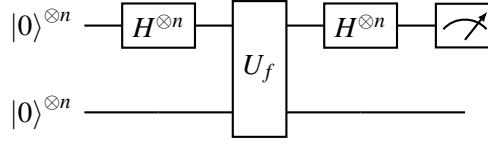
find s . (if $s = 0^n$, then f is one-to-one. otherwise, f is two-to-one.)

classically: requires $\Omega(2^{n/2})$ queries (birthday paradox argument).

quantumly: $O(n)$ queries and classical post-processing.

this is an exponential speedup — the first problem where quantum computing provably offers an exponential advantage.

6.2 The Circuit



6.3 Analysis

after hadamards:

$$\frac{1}{\sqrt{2^n}} \sum_x |x\rangle |0^n\rangle$$

after oracle:

$$\frac{1}{\sqrt{2^n}} \sum_x |x\rangle |f(x)\rangle$$

now, conceptually, suppose we measure the second register and get some value $f(z)$. then the first register collapses to a superposition of the (at most two) inputs that map to $f(z)$:

if $s \neq 0^n$:

$$\frac{1}{\sqrt{2}} (|z\rangle + |z \oplus s\rangle)$$

after hadamards on first register:

$$\frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2^n}} \sum_y [(-1)^{y \cdot z} + (-1)^{y \cdot (z \oplus s)}] |y\rangle \quad (11)$$

$$= \frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2^n}} \sum_y (-1)^{y \cdot z} [1 + (-1)^{y \cdot s}] |y\rangle \quad (12)$$

the factor $[1 + (-1)^{y \cdot s}]$ is:

- 2 if $y \cdot s = 0$
- 0 if $y \cdot s = 1$

so we only measure y values satisfying $y \cdot s = 0 \pmod{2}$.

6.4 Classical Post-Processing

each run of the circuit gives us one equation $y \cdot s = 0$. after $O(n)$ runs, we collect $n - 1$ linearly independent equations and solve the linear system over $\mathbb{F}_2$ (binary field) to find s .

Key Point. simon's algorithm structure:

1. repeat $O(n)$ times: run the quantum circuit, measure, get a random y satisfying $y \cdot s = 0$
2. collect all the y values into a matrix
3. solve the null space to find s (gaussian elimination over $\mathbb{F}_2$)

total: $O(n)$ quantum queries + $O(n^3)$ classical post-processing. exponentially better than the $O(2^{n/2})$ classical query complexity.

Example 6.1. $n = 3, s = 110$.

after several runs, suppose we collect:

$$y_1 = 001 : \quad 0 \cdot 1 + 0 \cdot 1 + 1 \cdot 0 = 0 \quad \checkmark \quad (13)$$

$$y_2 = 110 : \quad 1 \cdot 1 + 1 \cdot 1 + 0 \cdot 0 = 0 \quad \checkmark \quad (14)$$

$$y_3 = 111 : \quad 1 \cdot 1 + 1 \cdot 1 + 1 \cdot 0 = 0 \quad \checkmark \quad (15)$$

we need to find s such that $y_i \cdot s = 0$ for all i .

from the linear system (gaussian elimination): $s_1 = s_2, s_3 = 0$. nontrivial solution: $s = 110$.

6.5 Why Simon's Algorithm Matters

Intuition. simon's algorithm is historically important bcs it was the first to show an exponential quantum speedup for a well-defined problem. it directly inspired peter shor to develop his factoring algorithm, which is essentially a continuous version of simon's algorithm (replacing the XOR group structure with modular arithmetic).

the key insight from simon's algorithm: the hadamard transform converts the oracle's domain structure (the period s) into linear constraints on the measurement outcomes. this is a recurring theme in quantum algorithms.

7 Comparison of the Algorithms

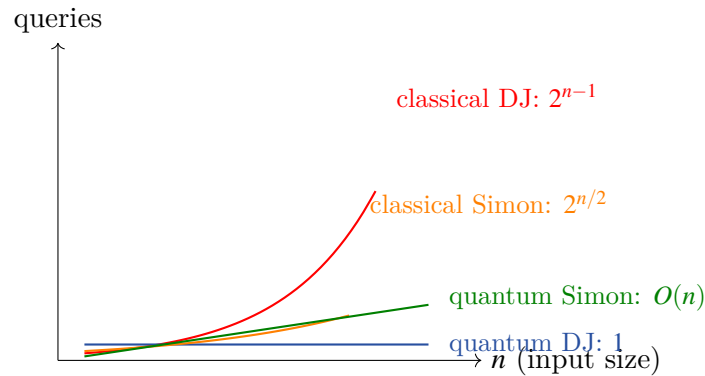
algorithm	problem	classical queries	quantum queries
deutsch	constant vs balanced ($n = 1$)	2	1
deutsch-jozsa	constant vs balanced (n bits)	$2^{n-1} + 1$	1
bernstein-vazirani	find secret string s	n	1
simon	find period s	$\Omega(2^{n/2})$	$O(n)$

Key Point. all four algorithms share the same structure:

1. hadamard: create superposition
2. oracle: encode function information into phases
3. hadamard: create interference
4. measure: read out the answer

the H-oracle-H sandwich is the fundamental quantum algorithm pattern. the difference between the algorithms is in the oracle structure and wt information the interference reveals.

heres a visual of how the speedups scale:



deutsch-jozsa gives an exponential speedup (the most dramatic kind). simon gives a superpolynomial speedup. both use the same H-oracle-H sandwich — the difference is in the oracle.

8 The Hadamard Transform and Fourier Analysis

the hadamard transform is doing smtg very specific: its the discrete fourier transform over $\mathbb{Z}_2^n$ (the group of n -bit binary strings under XOR).

$$H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle$$

this maps from the “position basis” to the “frequency basis” (just like the classical fourier transform). the oracle encodes information in the position basis, and the hadamard reveals the frequency-domain structure.

Intuition. the reason the H-oracle-H pattern works is bcs:

1. first H: express the input as a superposition in the position basis
2. oracle: the oracle modifies the amplitudes based on $f(x)$, encoding the function’s structure into the position-basis amplitudes
3. second H: fourier transform to the frequency basis, where the function’s global properties (constant vs balanced, period, secret string) become directly readable from the amplitudes

global properties of f become localized in the fourier domain. this is the deep reason quantum algorithms work.

8.1 The Boolean Fourier Expansion

any boolean function $f: \{0,1\}^n \rightarrow \{-1,1\}$ can be written as:

$$f(x) = \sum_{S \subseteq [n]} \hat{f}(S) \chi_S(x)$$

where $\chi_S(x) = (-1)^{\bigoplus_{i \in S} x_i}$ r the parity functions and $\hat{f}(S)$ r the fourier coefficients.

the deutsch-jozsa and bernstein-vazirani algorithms can be understood as measuring specific fourier coefficients of f — the quantum circuit is literally computing the fourier transform and reading off the coefficients.

9 Detailed Worked Examples

9.1 Deutsch-Jozsa with $n = 3$

Example 9.1. let $f: \{0,1\}^3 \rightarrow \{0,1\}$ be balanced, defined by $f(x) = x_1$ (output is just the first bit).

so $f(000) = 0, f(001) = 0, f(010) = 0, f(011) = 0, f(100) = 1, f(101) = 1, f(110) = 1, f(111) = 1$.

this is balanced: exactly 4 inputs give 0 and 4 inputs give 1.

the phase oracle: $O_f |x_1 x_2 x_3\rangle = (-1)^{x_1} |x_1 x_2 x_3\rangle$. this is just $Z \otimes I \otimes I$ acting on the first qubit.

step 1: $H^{\otimes 3} |000\rangle = |+++ \rangle$

step 2: $(Z \otimes I \otimes I) |+++ \rangle = |- \rangle |+\rangle |+\rangle$

(bcs $Z|+\rangle = |- \rangle$)

step 3: $H^{\otimes 3} |-+ + \rangle = |1\rangle |0\rangle |0\rangle = |100\rangle$

measure: $|100\rangle \neq |000\rangle$, so f is balanced. correct!

note: the measurement outcome 100 actually tells u that f depends on x_1 — the bernstein-vazirani algorithm extracts even more information from the same circuit.

9.2 Bernstein-Vazirani with $n = 4$

Example 9.2. secret string $s = 1011$.

$f(x) = x_1 \oplus x_3 \oplus x_4$ (inner product with 1011).

the oracle applies phase $(-1)^{x_1 \oplus x_3 \oplus x_4}$:

$$O_f = Z_1 \otimes I_2 \otimes Z_3 \otimes Z_4$$

(each qubit position where $s_i = 1$ gets a Z gate).

the circuit: $H^{\otimes 4} \cdot O_f \cdot H^{\otimes 4} |0000\rangle$

$$H^{\otimes 4} |0000\rangle = |++++ \rangle$$

$$O_f |++++ \rangle = Z_1 |+\rangle \otimes |+\rangle \otimes Z_3 |+\rangle \otimes Z_4 |+\rangle = |- \rangle |+\rangle |- \rangle |- \rangle$$

$$H^{\otimes 4} |-+ -- \rangle = |1\rangle |0\rangle |1\rangle |1\rangle = |1011\rangle$$

measure: $1011 = s$. done in one query.

9.3 Simon's Algorithm: Full 3-Qubit Example

Example 9.3. let $n = 2, s = 11$.

define f by: $f(00) = 01, f(01) = 10, f(10) = 10, f(11) = 01$.

check: $f(00) = f(11)$ and $00 \oplus 11 = 11 = s$. $f(01) = f(10)$ and $01 \oplus 10 = 11 = s$. good.

run 1:

$$|00\rangle |00\rangle \xrightarrow{H^{\otimes 2} \otimes I} \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle) |00\rangle$$

$$\xrightarrow{U_f} \frac{1}{2}(|00\rangle |01\rangle + |01\rangle |10\rangle + |10\rangle |10\rangle + |11\rangle |01\rangle)$$

$$= \frac{1}{2}[(|00\rangle + |11\rangle) |01\rangle + (|01\rangle + |10\rangle) |10\rangle]$$

if we measure the second register and get $|01\rangle$:

first register collapses to $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$

$$\begin{aligned} & \xrightarrow{H^{\otimes 2}} \frac{1}{\sqrt{2}}(|++\rangle + |--\rangle) \\ &= \frac{1}{\sqrt{2}} \cdot \frac{1}{2} [(|00\rangle + |01\rangle + |10\rangle + |11\rangle) + (|00\rangle - |01\rangle - |10\rangle + |11\rangle)] \\ &= \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \end{aligned}$$

measure: get either 00 or 11, each with probability 1/2.

if we get $y = 11$: check $y \cdot s = 1 \cdot 1 + 1 \cdot 1 = 0 \pmod{2}$. ✓

if we get $y = 00$: trivially $y \cdot s = 0$. (uninformative, need another run)

after enough runs collecting nontrivial y values, we solve to find $s = 11$.

10 Limitations and Context

Key Point. important caveats:

- these algorithms work in the oracle model — the speedup is relative to oracle queries, not total computation time
- in practice, constructing the oracle itself might require exponential resources, which would negate the query advantage
- deutsch-jozsa solves a problem that doesn't arise naturally in practice
- the randomized classical algorithm for deutsch-jozsa needs only $O(1)$ queries with bounded error — so the quantum advantage is only over deterministic classical algorithms
- simon's algorithm inspired shor's algorithm, which DOES solve a practically important problem (factoring)
- the techniques — superposition, interference, phase kickback, fourier analysis — show up everywhere in quantum computing

11 Summary

Key Point. wt to remember from this lecture:

1. the H-oracle-H sandwich is the fundamental pattern of quantum algorithms
2. phase kickback converts oracle queries into phase information
3. interference amplifies correct answers and cancels wrong ones
4. deutsch: constant vs balanced, 1 qubit, 1 query
5. deutsch-jozsa: constant vs balanced, n qubits, 1 query
6. bernstein-vazirani: find secret string, 1 query
7. simon's: find period, $O(n)$ queries (exponential speedup)

8. the hadamard transform is the fourier transform over $\mathbb{Z}_2^n$ — this is the mathematical reason these algorithms work

next up: QFT, phase estimation, shor's algorithm, and grover's search. these take the same principles and apply them to much more powerful settings.